

Таблицы: создание, изменение, удаление

```
// подключение к базе "sqlite"
Schema::connection('sqlite')
    ->create('users', function (Blueprint $table) {
        // ...
    });

if (! Schema::hasTable('users')) {
    Schema::create('users', function (Blueprint $table) {

        $table->engine = 'InnoDB'; // MySQL
        $table->charset = 'utf8mb4'; // MySQL
        $table->collation = 'utf8mb4_unicode_ci'; // MySQL

        $table->temporary(); // временная таблица (автом-и удалится при закрытии соединения)
        $table->comment('Business calculations'); // MySQL и Postgres

        // ...
    });
}

// если сущ. табл. "users" и в нем поле "email"
if (Schema::hasColumn('users', ['email', 'id'])) {

    // Обновление таблицы
    Schema::table('users', function (Blueprint $table) {
        $table->integer('votes');
    });
}

// Переименовать таблицу
Schema::rename($from, $to);

// Удалить таблицу
Schema::dropIfExists('users');
Schema::drop('users');
```

Столбцы

Перед изменением столбца необходимо установить doctrine/dbal

```
composer require doctrine/dbal
```

```
$table->id(); // псевдоним bigIncrements
$table->bigIncrements('id');
$table->mediumIncrements('id'); // автоматически увеличивающийся UNSIGNED MEDIUMINT
$table->increments('id'); // автоматически увеличивающийся UNSIGNED INTEGER
$table->smallIncrements('id'); // от -32768 до 32767. занимает 2 байта
```

```

$table->tinyIncrements('id'); // от -128 до 127, занимает 1 байт

$table->uuid('id');

// Индексы
$table->string('email')->unique();
$table->unique('email');
$table->index(['account_id', 'created_at']);
$table->unique('email', 'unique_email'); // unique_email - своё имя индекса
$table->primary('id');
$table->primary(['id', 'parent_id']); // составной ключ
$table->fullText('body'); // полнотекстовый индекс (MySQL/PostgreSQL)
$table->fullText('body')->language('english');
$table->spatialIndex('location'); // пространственный индекс (кроме SQLite)

$table->unsignedBigInteger('votes');
$table->unsignedMediumInteger('votes');
$table->bigInteger('votes');
$table->mediumInteger('votes');
$table->integer('votes');
$table->unsignedSmallInteger('votes'); // от 0 до 65535, занимает 2 байта
$table->smallInteger('votes');

$table->unsignedTinyInteger('votes'); // от 0 до 255, занимает 1 байт
$table->tinyInteger('votes');
$table->boolean('confirmed');

$table->binary('photo');

$table->tinyText('notes');
$table->char('name', 100);
$table->longText('description');
$table->text('description');
$table->mediumText('description');
$table->string('name', 100);

$table->timeTz('sunrise', $precision = 0); // TIME(с часовым поясом)
$table->time('sunrise', $precision = 0); // TIME
$table->dateTimeTz('created_at', $precision = 0);
.$table->dateTime('created_at', $precision = 0);
$table->date('created_at');
$table->nullableTimestamps(0); // псевдоним timestamps
$table->softDeletesTz($column = 'deleted_at', $precision = 0);
$table->softDeletes($column = 'deleted_at', $precision = 0);
$table->timestampTz('added_at', $precision = 0);
$table->timestamp('added_at', $precision = 0);
$table->year('birth_year');

// Дробный числа
$table->unsignedDecimal('amount', $precision = 8, $scale = 2);
$table->decimal('amount', $precision = 8, $scale = 2);

```

```
$table->double('amount', 8, 2);
$table->float('amount', 8, 2);

$table->foreignId('user_id');
$table->foreignIdFor(User::class);
$table->foreignUuid('user_id');

$table->ipAddress('visitor');
$table->macAddress('device');

$table->json('options');
$table->jsonb('options');

$table->enum('difficulty', ['easy', 'hard']);
$table->set('flavors', ['strawberry', 'vanilla']);

// Полиморфные связи
// добавляет 2 поля: {column}_id (UNSIGNED BIGINT) и {column}_type (VARCHAR)
$table->morphs('taggable'); // создаст 2 поля: "taggable_id" и "taggable_type"
$table->nullableMorphs('taggable');
$table->nullableUuidMorphs('taggable');
$table->uuidMorphs('taggable');

$table->rememberToken();

// Геометрия
$table->geometryCollection('positions');
$table->geometry('positions');
$table->lineString('positions'); // LINESTRING
$table->multiLineString('positions');
$table->multiPoint('positions');
$table->multiPolygon('positions');
$table->point('position');
$table->polygon('position');

$table->renameColumn('from', 'to');
$table->renameIndex('from', 'to')

$table->dropColumn('votes');
$table->dropColumn(['votes', 'avatar', 'location']);
$table->dropMorphs('morphable');
$table->dropRememberToken();
$table->dropSoftDeletes();
$table->dropSoftDeletesTz();
$table->dropTimestamps();
$table->dropTimestampsTz(); // Псевдоним dropTimestamps()
$table->dropPrimary('users_id_primary');
$table->dropUnique('users_email_unique');
$table->dropIndex('geo_state_index');
$table->dropIndex( ['state'] ); // Удалит индекс 'geo_state_index' (geo - имя таблицы)
$table->dropFullText('posts body fulltext');
```

```
$table->dropSpatialIndex('geo_location_spatialindex');
$table->dropForeign('posts_user_id_foreign');

// Внешний ключ
$table->unsignedBigInteger('user_id');
$table->foreign('user_id')->references('id')->on('users');

$table->foreignId('user_id')
    ->nullable()
    ->constrained()
    ->onUpdate('cascade')
    ->onDelete('cascade');

$table->cascadeOnUpdate();
$table->restrictOnUpdate();
$table->cascadeOnDelete();
$table->restrictOnDelete();
$table->nullOnDelete();

// Вкл./откл. внешних ключей в миграции
Schema::enableForeignKeyConstraints();
Schema::disableForeignKeyConstraints();
```

Модификаторы столбцов

```

->after('column')    // после столбца 'column' (MySQL)
->autoIncrement()    // как автоинкремент (первичный ключ)
->charset('utf8mb4')  // MySQL.
->collation('utf8mb4_unicode_ci') // параметры сортировки для столбца (MySQL / PostgreSQL / SQL
->comment('my comment') // комментарий к столбцу (MySQL / PostgreSQL)
->default($value)     // значение "по умолчанию"
->first()             // первым в таблице (MySQL)
->from($integer)      // начальное значение автоинкрементного поля (MySQL / PostgreSQL)
->invisible()         // "невидимый" для SELECT *запросов (MySQL)
->nullable($value = true) // Разрешить вставку нулевых значений
->storedAs($expression) // сохраненный сгенерированный столбец (MySQL / PostgreSQL)
->unsigned()          // ЦЕЛОЧИСЛЕННЫЕ столбцы (MySQL)
->useCurrent()        // CURRENT_TIMESTAMP в качестве значения по умолчанию
->useCurrentOnUpdate() // CURRENT_TIMESTAMP при обновлении записи
->virtualAs($expression) // виртуальный сгенерированный столбец (MySQL)
->generatedAs($expression) // столбец identity с указанными параметрами последовательности (Postg
->always()            // Определяет приоритет значений последовательности над входными данными для столбца и
->isGeometry()        // тип пространственного столбца

->default(new Expression('(JSON_ARRAY())')); // Выражения по умолчанию

->change()

```

#

App\Providers\AppServiceProvider:

```

public function boot()
{
    // настроить длину строки по умолчанию, генерируемую миграциями
    Schema::defaultStringLength(191);
}

```

События

```

Illuminate\Database\Events\MigrationsStarted    // Партия миграций собирается быть выполнена
Illuminate\Database\Events\MigrationsEnded      // Пакет миграций завершен
Illuminate\Database\Events\SchemaDumped        // Дамп схемы базы данных завершен
Illuminate\Database\Events\SchemaLoaded        // Загружен дамп существующей схемы базы данных

```